Journal of Computational and Applied Mathematics 486 (2026) 117652

Contents lists available at ScienceDirect

Journal of Computational and Applied Mathematics

journal homepage: www.elsevier.com/locate/cam

QUBO for training regression models

M. Caruso [a,b,c,d,*], D. Escanez-Exposito [e], P. Caballero-Gil [e], C. Kuchkovsky [d]

[a] UGR, Universidad de Granada, 18071, Granada, Spain
[b] FIDESOL, 18016, Granada, Spain
[c] UNIR, Universidad Internacional de La Rioja, Logroño, Spain
[d] QCentroid, 48001, Bilbao, Spain
[e] ULL, Universidad de La Laguna, 38200, Tenerife, Spain

ARTICLE INFO

Keywords:
QUBO
Machine learning
Regression model
Quantum training

ABSTRACT

A constantly increasing number of parameters in modern machine learning models has become a significant bottleneck in their efficient training, resulting in a disproportionate consumption of resources, expressed in terms of energy and computational time. The vast majority of such resources are allocated to the training phase rather than the inference stage, leading to an inefficient process [1–3]. This study addresses this problem by introducing a quantum alternative for training machine learning models, formalized as a Quadratic Unconstrained Binary Optimization (QUBO) problem. Adiabatic quantum computers have shown promise in approximately solving certain NP-hard problems, such as QUBO, potentially offering speed advantages over classical computers in specific scenarios [4–7]. Given that many machine learning problems are also NP-hard, adiabatic quantum computers could play a pivotal role in efficiently training machine learning models. Furthermore, inspired by the ones' complement binary numeral system, this work includes the proposal of an efficient binary-to-decimal encoding, which improves data representation in the quantum context. A concrete implementation of a regression training problem on specific quantum hardware is also presented here, the results of which suggest that the adoption of quantum techniques in model training allows not only to address the problem of parameter explosion but also to offer promising prospects for significantly accelerate the training process by exploiting the unique properties of quantum computation.

1. Introduction

The recent increase in the number of parameters of a given Machine Learning (ML) model presents a bottleneck in efficient training [8]. Comparatively, the training time is usually greater than the computation time, i.e., querying, of an already trained model [9]. In general, most resources, energy, computational power, and eventually time, for ML models are primarily used during the training stage [10].

In this document, an alternative approach is proposed for training ML regression models as a QUBO problem, accompanied by a practical implementation. An optimized binary-to-decimal encoding, inspired by the one's complement system, is introduced to enhance data representation in the quantum context. The proposed methodology is empirically validated through a concrete implementation on quantum hardware, demonstrating significant advantages over classical methods. Furthermore, this work includes

* Corresponding author.
E-mail addresses: mcaruso@ugr.es (M. Caruso), jescanez@ull.edu.es (D. Escanez-Exposito), pcaballe@ull.edu.es (P. Caballero-Gil), carlos@qcentroid.xyz (C. Kuchkovsky).

https://doi.org/10.1016/j.cam.2026.117652
Received 23 February 2024; Received in revised form 29 January 2025
Available online 1 April 2026
0377-0427/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

an original implementation that supports the proposed framework, utilizing a novel approach for representing real numbers with Boolean variables. The methodology provides a general recipe applicable to a specific model, which can be extended to other ML problems, such as classification and clustering e.g., support vector machines and balanced k-means, respectively.

Previous works, such as [11], have studied the possibility of realizing QUBO formulations for training ML models. However, in general, unlike this work, those are theoretical proposals without any empirical demonstration or implementation. Furthermore, in those works the used coding is a first approach that is optimized here.

Other publications, such as [12] and [13], show an extended study, with developments that support the theoretical framework. In order to implement a regression model, those papers use the same method to encode the data as the work mentioned above. In addition, they include original implementations to make interesting detailed comparisons between scikit-learn [14] and D-Wave [15,16]. As in those, this work also addresses the training of the simple case of regression models because an analytical solution is available that allows an objective comparison between obtained and previous results.

The present work is structured as follows. Section 2 includes a brief description of the type of models addressed throughout the work, what is meant by computing and training a model, and an introduction to regression models. In Section 3, QUBO type problems are introduced to train ML models. In order to transform real variable problems into binary variable problems, two ways to perform a binary-to-decimal encoding are introduced. An implementation and a discussion of those ideas are included in Section 4, using datasets of different sizes. Finally, this work concludes with a discussion presenting some findings and insights in the final Section 5.

2. Supervised ML model paradigm

Let $\mathtt{X}$ be a set of features of a given system, initially there are real numbers quantities. The goal is to predict another quantity, y, from a particular feature $\boldsymbol{x}\in\mathtt{X}$. The set of the predictions is denoted by $\mathtt{Y}$. These predictions may come from a deterministic or probabilistic parametric models, which refers to the kind of predictor being considered. In the first case, a function $f:\mathtt{X}\times\mathtt{P}\longrightarrow\mathtt{Y}$, predicts the value of y given a value of $\boldsymbol{x}$ and the parameters $\boldsymbol{q}\in\mathtt{P}$. In the second case, the features $\boldsymbol{x}$ and predictions y would be realizations of a random vector and a random variable, and there is a parameterized distribution of predictions conditioned on features, denoted generically as $P(y|\boldsymbol{x},\boldsymbol{q})$. Normally, it is considered that the random vector is composed of independently and identically distributed random variables.

In the realm of supervised learning, the need for accurately labeled data is essential. The primary emphasis of this work is on the training process, with less emphasis on validation and testing [18]. Let us denote by $\mathscr{D}$ the training dataset, consisting of N numerical feature vectors, taking values from a subset $\mathscr{X}\subset\mathtt{X}$ and a subset of labels, also numerical $\mathscr{Y}\subset\mathtt{Y}$, so that $\mathscr{D}=\mathscr{X}\times\mathscr{Y}$, which can be explicitly denoted as $\mathscr{D}=\{\langle\boldsymbol{x}_i,y_i\rangle\}_{i\in I_N}$, where $I_N:=\{1,\cdots,N\}\subset\mathbb{N}$.

There is a concise relationship between deterministic and probabilistic worlds. The elements of $\mathscr{D}$ would be realizations of a random vector $\boldsymbol{X}$ and a random variable Y, and there is a parameterized distribution of the conditioned random variable $Y|\boldsymbol{X}$ follows a distribution with a probability density, or mass function, $P(y|\boldsymbol{x},\boldsymbol{q})$. We need a function that quantifies how plausible it is that concrete realizations from $\mathscr{D}$ will be fit correctly to a specific model, given by that P. The well known likelihood function L is commonly used to build a distance measure between features and predictions, where $L(\boldsymbol{q})=\prod_{i\in I_N}P(y_i|\boldsymbol{x}_i,\boldsymbol{q})$, and $-\log L(\boldsymbol{q})$ is used as a distance measure. Another hypothesis largely used is to conceive as $Y=F(\boldsymbol{X},\boldsymbol{q})+\boldsymbol{\xi}$, where the additive fluctuation term $\boldsymbol{\xi}\sim\mathscr{N}(\boldsymbol{0},\boldsymbol{\sigma}^2)$ can be introduced to account for the variability of Y not explained by the relationship F, thus allowing for a more realistic representation of the observations. Under the foregoing conditions and normally distributed random variables, both deterministic and probabilistic parameter estimation methods yield the same results [17].

2.1. Training a ML model

In the parametric context, training a model means estimating the parameters $\boldsymbol{q}\in\mathtt{P}$ that minimize some kind of distance, $\varphi:\mathtt{P}\longrightarrow\mathbb{R}$, related to the model's prediction, using each element $\boldsymbol{x}_i\in\mathscr{D}$ and the corresponding value that arises from the proper labeling of $\mathscr{D}$, i.e., y_i as the true values. Therefore, training a parametric model involves solving the optimization problem

$$\arg\min_{\boldsymbol{q}\in\mathtt{P}}\ \varphi(\boldsymbol{q}) \tag{1}$$

Eventually, the above problem may have a collection of local minimal solutions, so there are different criteria to select one of them to terminate the training procedure [18]. Once the parameters $\boldsymbol{q}$ are fixed based on the dataset $\mathscr{D}$, the next step is the validation and testing, to finally use the trained model to predict y based on a new value $\boldsymbol{x}\notin\mathscr{X}$.

In this work, in order to tackle a training problem using quantum computation, a simple case is addressed that requires focusing attention only on the encoding of a QUBO type problem.

This work focuses on the deterministic predictor, defined by the function $y=f(\boldsymbol{x},\boldsymbol{q})$. In particular, the mean squared error can be used to determine the distance:

$$\varphi(\boldsymbol{q})\propto\frac{1}{N}\sum_{i\in I_N}\left[f(\boldsymbol{x}_i,\boldsymbol{q})-y_i\right]^2 \tag{2}$$

which is a function only of the parameters $\boldsymbol{q}$. Thus, the notion of the distance coming from $\varphi(\boldsymbol{q})$ can be expressed as the square of the $2-$norm of an $N-$dimensional vector, where it $i-$component is given by $f(\boldsymbol{x}_i,\boldsymbol{q})-y_i$. This involves that it could be said that if the elements of $\mathscr{D}$ are *well-labeled*, then the difference $f(\boldsymbol{x}_i,\boldsymbol{q})-y_i$ should be 0, but note that the predictor function f is still undefined in terms of its parameters $\boldsymbol{q}$. Therefore, the objective is to find those parameters $\boldsymbol{q}$ such that the average of that differences over all elements of $\mathscr{D}$ is minimized.

2.2. Regression model

The deterministic predictor $y{=}f(\boldsymbol{x},\boldsymbol{q})$ is defined by the expression:

$$f(\boldsymbol{x},\boldsymbol{q}) = \boldsymbol{q}\cdot\boldsymbol{\phi}(\boldsymbol{x}) = \sum_{n\in I_m} q_n\,\phi_n(\boldsymbol{x}) \tag{3}$$

where $\cdot$ denotes the inner product of vectors, and the number of different parameters and the number of components of the function $\boldsymbol{\phi}$ coincide, $\boldsymbol{q}{=}(q_1,\cdots,q_m)^{\mathrm{t}}$, $\boldsymbol{\phi}(\boldsymbol{x}){=}(\phi_1(\boldsymbol{x}),\cdots,\phi_m(\boldsymbol{x}))^{\mathrm{t}}$, t representing the matrix transposition. Thus, according to the general form (3), y depends on $\boldsymbol{x}$ through a function $\boldsymbol{\phi}\colon \mathtt{X}\longrightarrow\mathtt{Z}$ called *feature map* [18]. As this name suggests, $\boldsymbol{\phi}$ maps the system's features to a space of dimension equal to m, i.e., $\mathtt{Z}\subseteq\mathbb{R}^m$. One of the advantages of using $\boldsymbol{\phi}$ is that it is possible for the feature set $\mathtt{X}$ to have a dimension different from $\mathtt{Z}$, including the possibility of being infinitely dimensional.

A constant term b, know as additive bias, can be included in the expression (3) so that $f(\boldsymbol{x},\boldsymbol{q}) = b + \boldsymbol{q}\cdot\boldsymbol{\phi}(\boldsymbol{x})$, by simply extending the number of components of $\boldsymbol{q}$ and $\boldsymbol{\phi}(\boldsymbol{x})$ in a unit, so that $\boldsymbol{q}{=}(q_0,q_1,\cdots,q_m)^{\mathrm{t}}$ and $\boldsymbol{\phi}(\boldsymbol{x}){=}(\phi_0(\boldsymbol{x}),\phi_1(\boldsymbol{x}),\cdots,\phi_m(\boldsymbol{x}))^{\mathrm{t}}$ with $q_0 = b$ and $\phi_0(\boldsymbol{x}) = 1$. The expression (3) or its general version with additive bias b can be obtained as a particular case through a Taylor series expansion of $f(\boldsymbol{x},\boldsymbol{q})$ around $\boldsymbol{q}{=}\mathbf{0}$ and truncated to the nontrivial first order:

$$y = f(\boldsymbol{x},\mathbf{0}) + \boldsymbol{q}\cdot\partial_{\boldsymbol{q}} f(\boldsymbol{x},\boldsymbol{q})|_{(\boldsymbol{x},\mathbf{0})} + \cdots \tag{4}$$

Note that the term *linear* in the concept of regression does not refer to the functional relationship between y and $\boldsymbol{x}$ but rather to the functional relationship between y and $\boldsymbol{q}$, which must be linear. The function f may not be linear with respect to $\boldsymbol{x}$; however, we can always deal with an abstract problem $g(\boldsymbol{z},\boldsymbol{q})$ that is linear with $\boldsymbol{z}$ so that from a feature map $\boldsymbol{\phi}$, such that $\boldsymbol{z}=\boldsymbol{\phi}(\boldsymbol{x})$, we can recover f from g as $f(\boldsymbol{x},\boldsymbol{q}) = g(\boldsymbol{\phi}(\boldsymbol{x}),\boldsymbol{q})$.

Regarding the distance φ, to provide a compact expression for it, a matrix $\boldsymbol{\Phi}\in\mathbb{R}^{m\times N}$ can be defined by arranging each vector of $\{\boldsymbol{\phi}(\boldsymbol{x}_i)\}_{i\in I_N}$ as rows. Explicitly, $\boldsymbol{\Phi}$ is the matrix defined using the feature map evaluated on each element $\boldsymbol{x}_i$ of the training dataset $\mathscr{D}$:

$$\boldsymbol{\Phi} = \Big(\boldsymbol{\phi}(\boldsymbol{x}_1)\cdots\boldsymbol{\phi}(\boldsymbol{x}_i)\cdots\boldsymbol{\phi}(\boldsymbol{x}_N)\Big) = \begin{pmatrix} \phi_1(\boldsymbol{x}_1) & \cdots & \phi_1(\boldsymbol{x}_i) & \cdots & \phi_1(\boldsymbol{x}_N)\\ \vdots & & \vdots & & \vdots\\ \phi_m(\boldsymbol{x}_1) & \cdots & \phi_m(\boldsymbol{x}_i) & \cdots & \phi_m(\boldsymbol{x}_N)\end{pmatrix} \tag{5}$$

where $\boldsymbol{\Phi}$ is called feature matrix, whose elements are $\Phi_{ij}{=}\phi_i(\boldsymbol{x}_j)$. Note that the literature usually defines $\boldsymbol{\Phi}$ by arranging each $\boldsymbol{\phi}(\boldsymbol{x}_i)$ as rows of the feature matrix [18]. This involves defining the column vector $\boldsymbol{y}\in\mathbb{R}^N$ for the labels of $\mathscr{D}$, $\boldsymbol{y}=(y_1,\cdots y_N)^{\mathrm{t}}$. In this way, the distance φ can be expressed in the following form, which is obtained as a particular case of (2):

$$\varphi(\boldsymbol{q}) = \sum_{i\in I_N}\big(\boldsymbol{q}\cdot\boldsymbol{\phi}(\boldsymbol{x}_i) - y_i\big)^2 \tag{6}$$

and the optimization problem (1) has a unique and analytical solution $\boldsymbol{q}_*$ given by

$$\boldsymbol{q}_*{=}(\boldsymbol{\Phi}\boldsymbol{\Phi}^{\mathrm{t}})^{-1}\boldsymbol{\Phi}\boldsymbol{y} \tag{7}$$

which requires the existence of the inverse of the square matrix $\boldsymbol{\Phi}\boldsymbol{\Phi}^{\mathrm{t}}$. In fact, coefficient estimates hinge on the independence of features. If features are correlated and the rows of the matrix $\boldsymbol{\Phi}$ exhibit an almost linear dependence, the design matrix approaches singularity [19]. Consequently, the solution (7) becomes exceptionally sensitive to random errors in the observed target, leading to a significant increase in variance.

3. Training from QUBO

Note that the predictor function f is a linear function of the parameters while the distance function φ is quadratic in the parameters. Since the aim of this work is to map QUBO-type problems to the distance function φ, it is necessary to explicitly write down all the terms of the function φ that is intended to be optimised. Thus, the optimization problem for training is a quadratic function that can be expressed as:

$$\varphi(\boldsymbol{q}) = \sum_{(i,j)\in I_m^2}\Lambda_{ij}q_iq_j - \sum_{i\in I_m}2p_iq_i + c \tag{8}$$

where the matrix $(\Lambda_{ij}) =: \boldsymbol{\Lambda}\in\mathbb{R}^{m\times m}$, the column vector $(p_1,\cdots,p_m)^{\mathrm{t}} =: \boldsymbol{p}\in\mathbb{R}^m$ and the scalar $c = \boldsymbol{y}\cdot\boldsymbol{y}$ is fixed.

Given that φ is a convex function, then the stationary point $\boldsymbol{q}_*$, obtained from $\nabla\varphi(\boldsymbol{q}_*) = 0$, is also the minimum:

$$\boldsymbol{q}_*{=}\boldsymbol{\Lambda}^{-1}\boldsymbol{p} \tag{9}$$

The connection between the objective functions given by (6) and (8) and its solutions (7) and (9) is manifested from $\boldsymbol{\Lambda}{=}\boldsymbol{\Phi}\boldsymbol{\Phi}^{\mathrm{t}}$ and $\boldsymbol{p}{=}\boldsymbol{\Phi}\boldsymbol{y}$. The value of the additive constant c of does not affect the value of $\boldsymbol{q}_*$, this is because the set (1) is invariant under the translation $\varphi(\boldsymbol{q})\longmapsto\varphi(\boldsymbol{q})+c$ for all fixed value $c\in\mathbb{R}$.

Let us now consider the QUBO-type problems $\boldsymbol{u}{\in}\mathtt{B}$, $\boldsymbol{Q}{\in}\mathbb{R}^{n\times n}$, $\boldsymbol{w}{\in}\mathbb{R}^n$ and an objective function $f(\boldsymbol{u}) = \boldsymbol{u}\cdot\boldsymbol{Q}\,\boldsymbol{u} - \boldsymbol{w}\cdot\boldsymbol{u}$, which contains quadratic terms and linear terms. Again, additive constant terms are not considered in f since the result is the same for the optimization problem at hand. However, it is possible to reduce the linear part into a quadratic part, using that $u_i^2 = u_i$, $\forall i\in I_n$. Note that $f(\boldsymbol{u})$ can

also be written as $f(\boldsymbol{u}) = \boldsymbol{u} \cdot [\boldsymbol{Q} - diag(\boldsymbol{w})]\boldsymbol{u}$, where $\texttt{diag}(\boldsymbol{w})$ represents the matrix that has on its main diagonal the components of $\boldsymbol{w}$ and null the rest of its matrix elements. The function f is redefined as:

$$f(\boldsymbol{u}) := \boldsymbol{u} \cdot \boldsymbol{Q}_{\#}\boldsymbol{u}, \tag{10}$$

where $\boldsymbol{Q}_{\#} = \boldsymbol{Q} - \texttt{diag}(\boldsymbol{w})$. The corresponding QUBO-problem associated to (1) is the minimization problem:

$$\arg\min_{\boldsymbol{u}\in\mathbb{B}} f(\boldsymbol{u}) \tag{11}$$

Given finite memory and processing time requirements, the interest is in finding a mapping between a particular subset of real numbers, those described with maximal finite precision. Each real number q can be approximated as a finite sum $q = \pm\sum_{k=-\alpha}^{\beta} a_k 10^k = \pm a_\beta a_{\beta-1}\cdots a_0 \cdot a_{-1}\cdots a_{-\alpha}$, where $a_k \in \{0,\cdots,9\}$ and $\alpha+\beta$ is the precision. It can be said that $a_\beta a_{\beta-1}\cdots a_0$ represents the integer part of q and $a_{-1}\cdots a_{-\alpha}$ the fractional (rational) part of q, because this number is given by $a_{-1}\cdots a_{-\alpha}/10^n$, which is a rational number.

Now the connection between the two optimization problems (1) and (11) is addressed. For this it is necessary to find an encoding that allows us to map the real vector $\boldsymbol{q}$ into the binary vector $\boldsymbol{u}$. The idea is to find a linear transformation $\boldsymbol{T}$ such that $\boldsymbol{q}=\boldsymbol{T}\boldsymbol{u}$ thus:

$$\varphi(\boldsymbol{T}\boldsymbol{u}) =: f(\boldsymbol{u}) \tag{12}$$

From φ of (8), the corresponding matrix $\boldsymbol{Q}$ and the vector $\boldsymbol{w}$ in the function f from (12) are given by

$$\boldsymbol{Q} = \boldsymbol{T}^{\mathrm{t}}\boldsymbol{\Lambda}\boldsymbol{T}, \quad \boldsymbol{w} = 2\boldsymbol{T}^{\mathrm{t}}\boldsymbol{p}, \tag{13}$$

from (10) the matrix $\boldsymbol{Q}_{\#}$ can be obtained from (13).

In the following sections, two methods for performing this binary to decimal encoding and finding a linear transformation for each method will be examined. The first encoding is documented in [11], although no implementations were made in that work. Additionally, another more efficient coding is here introduced with its corresponding implementations.

3.1. Encoding #1

Considering now a binary representation of each $q\in\mathbb{R}$, there exist a binary string $(u_1,\cdots,u_k,\cdots,u_K)\in\{0,1\}^K$ and a frame vector $\boldsymbol{\varrho} = (\varrho_1,\cdots,\varrho_k,\cdots,\varrho_K)$, where ϱ_k can be negative or positive and its absolute value is an integer power of 2, such that

$$q = \sum_{k=1}^{K} u_k\varrho_k \Big/ |\varrho_k| = 2^{s_k}, s_k \in \mathbb{Z}. \tag{14}$$

If $S=\{s_k\}_{k\in I_K}$ is an ordered series of consecutive powers of 2, $-a=\min(S)$, $b=\max(S)$. There is an equal number of negatives and positive values of ϱ_k, then $K = 2(b+a+1)$. In this case the maximum decimal number that can be represented with this encoding is given by $M_\varrho = \sum_{k\in I_{K^+}}\varrho_k$, where $I_{K^+} = \{k\in I_K : \varrho_k > 0\}$ thus $M_\varrho = 2^{b+1} - 2^{-a}$, by symmetry $-M_\varrho$ is also the minimum decimal number that can be represented.

Thus a given vector $\boldsymbol{q}$ of $m-$decimal components, there is matrix $\boldsymbol{U}$ where each row is a binary string $(u_{i1},\cdots,u_{ik},\cdots,u_{iK})=\boldsymbol{u}_i\in\{0,1\}^K$, we have

$$\boldsymbol{q}=\boldsymbol{U}\boldsymbol{\varrho}, \quad \boldsymbol{U}=\begin{pmatrix}\boldsymbol{u}_1\\ \vdots \\ \boldsymbol{u}_m\end{pmatrix} \tag{15}$$

can also be written as

$$\boldsymbol{q} = \left(\mathbf{I}_m \otimes \boldsymbol{\varrho}\right)\mathrm{vec}(\boldsymbol{U}) \tag{16}$$

where $\mathbf{I}_m$ is the $(m\times m)-$identity matrix, $(\mathbf{I}_m\otimes\boldsymbol{\varrho})\in\mathbb{R}^{m\times m\cdot K}$, $\boldsymbol{U}\in\{0,1\}^{m\times K}$ and $\mathrm{vec}(\boldsymbol{U})\in\{0,1\}^{m\cdot K}$. The expression (16) is precisely our objective, since it comprises a way to encode a real vector $\boldsymbol{q}$ from a boolean vector $\boldsymbol{u} = \mathrm{vec}(\boldsymbol{U})$ through a linear transformation, $\boldsymbol{T} = \mathbf{I}_m\otimes\boldsymbol{\varrho}$, which depends on the frame vector $\boldsymbol{\varrho}$. The matrix $\boldsymbol{Q}\in\mathbb{R}^{m\cdot K\times m\cdot K}$ and the vector $\boldsymbol{w}\in\mathbb{R}^{m\cdot K}$ are obtained from (13) and the obtained linear transformation $\boldsymbol{T}$.

The decimal range that can be represented using the encoding #1 is given by the symmetric interval $[-M_\varrho, M_\varrho]$, characterized by M_ϱ known as decimal representation capability. Since the original problem (1) has m variables, the number n_1 of the involved boolean variables for QUBO problem (11) is equal to $m\cdot K$. Lastly using (10) the matrix $\boldsymbol{Q}_{\#}$ can be obtained from $\boldsymbol{Q}$ and $\texttt{diag}(\boldsymbol{w})$.

3.2. Encoding #2

There is another representation of decimal number $q\in\mathbb{R}$ in terms of $L+1$ binary string, which encodes the sign in a single bit, and the remaining L bits are used to denote the integer part of q and the fractional part of q. This encoding is based on one's complement.

Considering the binary representation of each $q\in\mathbb{R}$, there exist a binary string $(u_0,u_1,\cdots,u_l,\cdots,u_L)\in\{0,1\}^{L+1}$ and frame vector $\boldsymbol{\varrho} = (\varrho_1,\cdots,\varrho_l,\cdots,\varrho_L)$, where in this case ϱ_l is strictly positive power or 2. The boolean variable u_0 encodes the sign of q, and the string of the remaining L bits are used to denote the integer part of q and the fractional part of q. In order to obtain a linear encoding and knowing that the positive decimal numbers can be encoded as $\sum_{l=1}^{L}\varrho_l u_l$, it can be write q as

$$q = -\mathfrak{s}\sum_{l=1}^{L}\varrho_l + \sum_{l=1}^{L}\varrho_l u_l \Big/ \varrho_l = 2^{t_l}, t_l\in\mathbb{Z}. \tag{17}$$

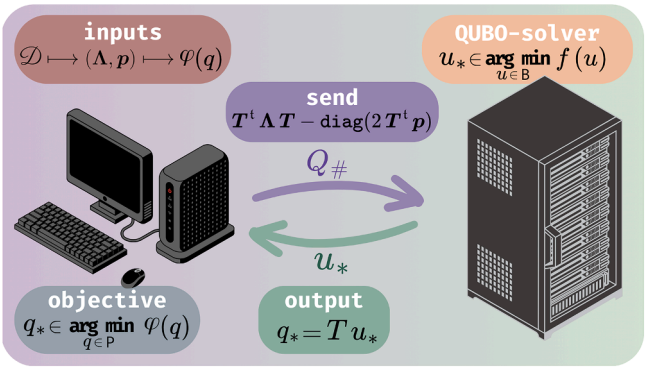


Fig. 1. Training scheme for a ML model through the resolution of a QUBO problem.

Note that $\mathfrak{s} = 0, 1$ is the bit that codifies the sign of q. If $T = \{t_l\}_{l \in I_L}$ is an ordered series of consecutive powers of 2, $-\alpha$=min(T), β=max(T) and L are related through $L = \beta + \alpha + 1$. In this case the maximum decimal number that can be represented with this encoding is given by $N_\varrho = \sum_{l=1}^{L} \varrho_l$ thus $N_\varrho = 2^{\beta+1} - 2^{-\alpha}$ and by symmetry $-N_\varrho$ is also the minimum decimal number that can be represented.

The expression (17) can be rewritten as $q = \sum_{l=0}^{L} u_l \varrho_l$, where $u_0 := \mathfrak{s}$ is the sign bit of q and $\varrho_0 := -N_\varrho$. Thus a given vector $\boldsymbol{q} \in \mathbb{R}^m$, a linear mapping can be constructed, but now the *extended* frame vector $(\varrho_0, \cdots, \varrho_L)$, which includes ϱ_0, and $(u_{j0}, u_{j1}, \cdots, u_{jl}, \cdots, u_{jL}) = \boldsymbol{u}_j \in \{0,1\}^{L+1}$, where $u_{j0} = \mathfrak{s}_j$ is the sign bit of q_j and $\{u_{j1}, \cdots, u_{jl}, \cdots, u_{jL}\}$ is the unsigned representation of q_j, the integer and the fractional parts of q_j. A similar expression to (15) is utilized, wherein an additional matrix $\boldsymbol{U} \in \{0,1\}^{m \times (L+1)}$ is introduced. This matrix consists of rows, each representing a binary string $\boldsymbol{u}_j$ of $L+1$ elements. Also, another frame vector is incorporated, denoted as its *extended* version $(\varrho_0, \cdots, \varrho_L)$. In this manner, for $\boldsymbol{q}$, it is established that

$$\boldsymbol{q} = \left[\mathbf{I}_m \otimes (-N_\varrho, \boldsymbol{\varrho})\right] \mathrm{vec}(\boldsymbol{U}), \tag{18}$$

where $[\mathbf{I}_m \otimes (-N_\varrho, \boldsymbol{\varrho})] \in \mathbb{R}^{m \times m \cdot (L+1)}$, and $\mathrm{vec}(\boldsymbol{U}) \in \{0,1\}^{m \cdot (L+1)}$, note that in this case $\boldsymbol{U}$ has $L+1$ columns. The expression (18) is precisely our objective, since it comprises a way to encode a real vector $\boldsymbol{q}$ from a boolean vector $\boldsymbol{u} = \mathrm{vec}(\boldsymbol{U})$ through a linear transformation, $\boldsymbol{T} = \mathbf{I}_m \otimes (-N_\varrho, \boldsymbol{\varrho})$, which depends on the frame vector $\boldsymbol{\varrho}$. The matrix $\boldsymbol{Q} \in \mathbb{R}^{m \cdot (L+1) \times m \cdot (L+1)}$ and the vector $\boldsymbol{w} \in \mathbb{R}^{m \cdot (L+1)}$ are obtained from (13) and the obtained linear transformation $\boldsymbol{T}$.

The decimal range that can be represented using the encoding #2 is given by the symmetric interval $[-N_\varrho, N_\varrho]$, characterized by N_ϱ, i.e. decimal representation capability. Since the original problem (1) has m variables, the number n_2 of the involved boolean variables for QUBO problem (11) is equal to $m \cdot (L+1)$. Lastly using (10) the matrix $\boldsymbol{Q}_\#$ can be obtained from $\boldsymbol{Q}$ and $\mathtt{diag}(\boldsymbol{w})$.

4. Implementation and discussion

In this section, an implementation conducted in python will be demonstrated. The D-Wave hardware provider [20] has been employed as a QUBO-solver in this context. The choice of using D − Wave's quantum annealing hardware for this implementation was motivated by its demonstrated efficiency in solving QUBO problems with high precision. Is one of the most widely used quantum libraries, which includes a collection of tools and modules for programming and interacting with quantum computers and simulators. Compared to classical solvers, quantum hardware offers potential advantages in terms of scalability and computational speed, although current limitations in qubit connectivity and noise remain challenges. Regarding related work, while studies such as [11,12] explored QUBO formulations for machine learning models, they primarily focused on theoretical approaches without concrete implementations. Our work extends these efforts by providing an empirical demonstration and introducing an optimized binary-to-decimal encoding, which enhances data representation and reduces resource requirements.

These systems are designed to solve QUBO problems through quantum annealing. The upcoming Fig. 1 illustrates the training scheme for a machine learning model achieved through solving an specific QUBO problem. The encodings #1 and #2 are represented in the binary - decimal map from (16) or (18), respectively. The procedure involves constructing and sending the matrix $\boldsymbol{Q}_\#$ to the hardware provider, receiving $\boldsymbol{u}_*$, and transforming it to obtain the solution of (1) $\boldsymbol{q}_*$ using $\boldsymbol{T}$. The general idea is to prepare the real variable (decimal) optimisation problem associated with the ML model training, then construct a mapping from the decimal problem to a binary variable problem and use a QUBO hardware problem provider to solve it in the binary context. Finally use the binary-to-decimal mapping to obtain the solution.

Algorithm A starts by extracting labels from a dataset, forming a label vector $\boldsymbol{y}$, which represents the target values of the ML model. Next, it computes a feature matrix $\boldsymbol{\Phi}$ by applying a feature map $\boldsymbol{\phi}$ to each data point $\boldsymbol{x}$ in the dataset. This feature map transforms the input data into a higher-dimensional feature space. Then it must be calculated the necessary elements for the optimization process, including the matrix $\boldsymbol{\Lambda}$ and vector $\boldsymbol{p}$, which are used in the objective function $\varphi(\boldsymbol{q})$, depending on the chosen encoding method (either 1 or 2), the construction a binary-decomposed mapping matrix $\boldsymbol{T}$ that maps the parameters of the ML model to binary variables. Using $\boldsymbol{T}$, a QUBO problem is formulated. It then sends this problem to a QUBO−solver, which finds the locally optimal binary vector $\boldsymbol{u}_*$ that

minimizes the objective function. Finally, the output $\boldsymbol{q}_*$ is obtained by multiplying the optimal binary vector $\boldsymbol{u}_*$ with the binary-decomposed mapping matrix $\boldsymbol{T}$. This output represents the optimized parameters of the ML model, meeting the defined objective of minimizing the distance function $\varphi(\boldsymbol{q})$ over the parameter space P. The implementation follows the proposal algorithm written in pseudocode showed in Algorithm A.

Algorithm A: training a ML model from QUBO [pseudocode].

inputs: $y = \boldsymbol{q}\cdot\boldsymbol{\phi}(\boldsymbol{x})$ //ML model
$\mathscr{D} = \{\boldsymbol{x}_i, y_i\}_{i\in I_N}$ //data set
$\varphi(\boldsymbol{q}) = \sum_{i\in I_N}\left(\boldsymbol{q}\cdot\boldsymbol{\phi}(\boldsymbol{x}_i) - y_i\right)^2$ //distance

objective: $\boldsymbol{q}_* \in \underset{\boldsymbol{q}\in \mathrm{P}\subseteq\mathbb{R}^m}{\arg\min}\ \varphi(\boldsymbol{q})$ //solve (1)

$\boldsymbol{y}$−label vector
- $\boldsymbol{y} = (y_1, \cdots, y_N) \in \mathbb{R}^N$ //labels from $\mathscr{D}$

$\boldsymbol{\Phi}$−feature matrix
- $\boldsymbol{\phi} : \mathrm{X} \to \mathrm{Z} \subseteq \mathbb{R}^m$ //feature map
- $\boldsymbol{\Phi} = (\boldsymbol{\phi}(\boldsymbol{x}_1), \cdots, \boldsymbol{\phi}(\boldsymbol{x}_N)) \in \mathbb{R}^{m\times N}$ //feature matrix (5)

φ−elements
- $\boldsymbol{\Lambda} = \boldsymbol{\Phi}\boldsymbol{\Phi}^{\mathrm{t}} \in \mathbb{R}^{m\times m}$
- $\boldsymbol{p} = \boldsymbol{\Phi}\boldsymbol{y} \in \mathbb{R}^m$

$\boldsymbol{T}$−bin.dec map
- **case** (enc. = 1) **do**
  - $\boldsymbol{\varrho} = (\cdots, \varrho_k, \cdots) \in \mathbb{R}^K$ //frame vector
  - $\boldsymbol{T} = \mathbf{I}_m \otimes \boldsymbol{\varrho}$ //encoding #1 (16)
- **case** (enc. = 2) **do**
  - $\boldsymbol{\varrho} = (\cdots, \varrho_l, \cdots) \in \mathbb{R}^L,\ N_\varrho = \sum_{l=1}^{L}\varrho_l$ //ext. frame vector
  - $\boldsymbol{T} = \mathbf{I}_m \otimes (-N_\varrho, \boldsymbol{\varrho})$ //encoding #2 (18)

QUBO − solver
- $\boldsymbol{Q}_{\#} = \boldsymbol{T}^{\mathrm{t}}\boldsymbol{\Lambda}\boldsymbol{T} - \mathrm{diag}(2\,\boldsymbol{T}^{\mathrm{t}}\boldsymbol{p})$ //send to QUBO-solver
- $\boldsymbol{u}_* \in \underset{\boldsymbol{u}\in \mathrm{B}}{\arg\min}\ \boldsymbol{u}\cdot\boldsymbol{Q}_{\#}\boldsymbol{u}$ //receive from QUBO-solver

output $\boldsymbol{q}_* = \boldsymbol{T}\,\boldsymbol{u}_*$ //output meets the objective

The pseudocode presented in Algorithm A outlines the optimization problem of a real variable function associated with training a regression model using the QUBO formulation. First, labels are extracted from the dataset $\mathscr{D}$ to form the label vector $\boldsymbol{y}$. Next, the feature matrix $\boldsymbol{\Phi}$ is computed by applying a feature map function $\boldsymbol{\phi}$ to each data point $\boldsymbol{x}$. Then, the necessary elements for optimization are calculated, including the matrix $\boldsymbol{\Lambda}$ and the vector $\boldsymbol{p}$, which are used in the objective function $\varphi(\boldsymbol{q})$. Depending on the chosen encoding method (encoding #1 or #2), a binary-to-decimal mapping matrix $\boldsymbol{T}$ is constructed, transforming the model's parameters into binary variables. Using $\boldsymbol{T}$, a QUBO problem is formulated and sent to a QUBO solver, which finds the optimal binary vector $\boldsymbol{u}_*$ that minimizes the objective function. Finally, the output $\boldsymbol{q}_*$ is obtained by multiplying the optimal binary vector $\boldsymbol{u}_*$ with the mapping matrix $\boldsymbol{T}$, resulting in the optimized parameters of the machine learning model.

Due to the fact that the size of the underlying training problem (1) does not depend on the size of the dataset $N = |\mathscr{D}|$ but on the number of parameters m of the ML model, we will first work with a dataset $\mathscr{D}_1$ such that $|\mathscr{D}_1|$=3 to test the ideas. Subsequently, we will do the same with a dataset $\mathscr{D}_2$ such that $|\mathscr{D}_2|$=300.

A model has been implemented using the predictor $f_1(\boldsymbol{q}, x)$=q_0+$q_1 x$, where $\boldsymbol{q}$=(q_0, q_1) and (α, β)=$(2, 0)$ for the extended frame vector $(-N_\varrho, \boldsymbol{\varrho})$. From a small dataset $\mathscr{D}_1$={$(0, 0.5), (-1, 3), (-2, 4)$}. We obtain the parameters (q_0, q_1)=$(0.75, -1.75)$ which exactly match those obtained from the analytical solution (9) or even a classical implementation of the regression problem using [14]. The results are presented in Fig. 2.

Finally, from a dataset of size N=300, we have implemented a model given by the predictor $f_2(\boldsymbol{q}, x)$=$q_0 + q_1\sin(2\pi x) + q_2\sin(4\pi x) + q_3\sin(6\pi x)$, where $\boldsymbol{q}$=(q_0, q_1, q_2, q_3), (α, β)=$(3, 1)$ for the extended frame vector. In this case, a synthetic dataset $\mathscr{D}_2$={(x_k, y_k)}$_{k\in I_N}$ is considered, where each pair generated from x_k=k/N, $N = 300$ and the corresponding y_k=0.5+$0.25\sin(2\pi x_k)$+$0.125\sin(4\pi x_k)$+$0.125\sin(6\pi x_k)$+ξ_k, where $\xi_k \sim \mathcal{U}(-\epsilon, \epsilon)$ is a uniform random variable and ϵ=10^{-1}. We obtain the parameters (q_0, q_1, q_2, q_3)=$(0.5, 0.25, 0.125, 0.125)$ which exactly match those obtained from the analytical solution (9) or even a classical implementation of the regression problem using [14]. The results are showed in Fig. 3.

The respective number of boolean variables required for encodings #1 and #2 are n_1=$m\cdot K$ and n_2=$m\cdot(L+1)$. If the decimal range of the encodings #1 and #2 are similar, i.e. both have a similar decimal representation capability $M_\varrho \sim N_\varrho$, then $K \sim 2\,L$, and $n_1 \sim 2\,n_2$, i.e. encoding #2 requires about half as many boolean variables as encoding #1. Conversely if the required number of boolean

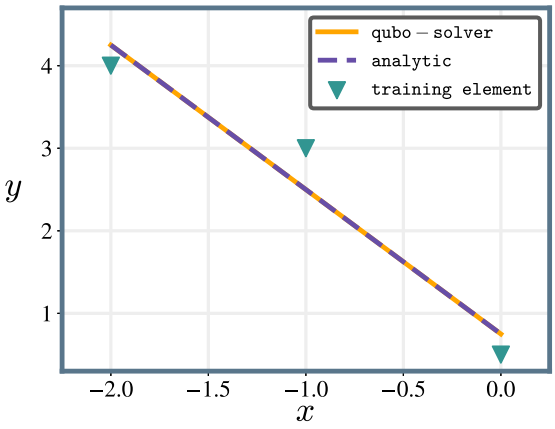


Fig. 2. The implementation of $f_1(\boldsymbol{q}, x)=q_0 + q_1 x$ using $\mathscr{D}_1$.

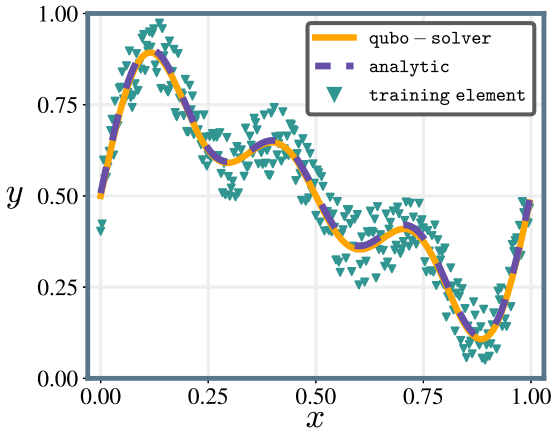


Fig. 3. The implementation of $f_2(\boldsymbol{q}, x)=q_0 + q_1 \sin(2\pi x) + q_2 \sin(4\pi x) + q_3 \sin(6\pi x)$ using $\mathscr{D}_2$.

variables of the encodings `#1` and `#2` are similar $n_1 \sim n_2$ then $K \sim L$ and $N_\varrho \sim M_\varrho\, 2^{L/2}$, i.e. the decimal representation capability of the encoding `#2` is much bigger than the encoding `#1`.

Note that, in both cases linear and non linear, the implementation of encoding `#1` or encoding `#2` arrives to the same results. Nevertheless, the results in the Figs. 2 and 3 are obtained from encoding `#2`.

A quick inspection of the analytical expression (9) for regression implies that it is necessary to construct the $\boldsymbol{\Lambda}$ matrix and invert it thus it time complexity class is $\mathscr{O}(m^3)$, then construct the vector $\boldsymbol{p}$ which belong to $\mathscr{O}(mN)$ and perform the matrix multiplication. Thus, the time complexity class of this analytical process would be $\mathscr{O}(m^3 N)$. On the other hand, using a singular value decomposition, the practical regression problem belongs to $\mathscr{O}(m^2 N)$ classically [14].

Denoting by ℓ as the size of the frame vector $\boldsymbol{\varrho}$ for encoding `#1` ($\ell=K$) or the size of the extended frame vector $(-N_\varrho, \boldsymbol{\varrho})$ for encoding `#2` ($\ell=L+1$), we analyze the time complexity of our approach in five steps:

1. φ objective function: $\mathscr{O}(m^2 N)$ for $\boldsymbol{\Lambda}$ and $\mathscr{O}(mN)$ for $\boldsymbol{p}$
2. $\boldsymbol{T}$ binary-decimal map: $\mathscr{O}(\ell m^2)$
3. $\boldsymbol{Q}_\#$ hardware embedding: $\mathscr{O}(\ell^2 m^2)$
4. $\boldsymbol{u}_*$ theoretical quantum annealing: $\mathscr{O}(e^{\sqrt{\ell m}})$
5. $\boldsymbol{q}_*$ output solution: $\mathscr{O}(\ell^2 m^2)$

Note that quantum annealing's theoretical time complexity is $\mathscr{O}(e^{\sqrt{\ell m}})$, while its classical counterpart is $\mathscr{O}(e^{\ell m})$, but practical estimates often rely on metrics such as ST99 and ST99(OPT), which provide the anticipated number of iterations needed to achieve a specific level of optimality with 99% certainty [25]. Adiabatic quantum computers accommodate finite-sized problems, e.g., `D-Wave` 2000Q can accommodate problems having 64 or fewer binary variables requiring all-to-all connectivity [5]. For such problems, constant annealing time and repetitions suffice. Thus, the total time complexity for linear regression problem formulated as an underlying `QUBO` problem, eventually implemented on an adiabatic quantum computer, is $\mathscr{O}(\ell^2 m^2 N)$. Although seemingly worse than classical time, considering fixed precision (ℓ as constant) thus time complexity is $\mathscr{O}(m^2 N)$, the same as the classical approach.

A brief comment about the expressivity of the model, the predictor

$$f(\boldsymbol{q}, x) = \sum_{n \in I \subset \mathbb{Z}} q_n \phi_n(x), \tag{19}$$

can be treated as a truncated series, where $\{\phi_n(x)\}_{n\in I}$ is a collection of special functions, i.e. Chebyshev, Laguerre and Hermite polynomials, Fourier functions, among many others [21]. Each of this class of functions is also a complete orthonormal basis of a vector space $L^2(D)$, where $D \subseteq \mathbb{R}$ is a certain domain, depending on which function class is chosen [22]. For example, for Chebyshev $D = [-1, 1]$, Laguerre $D = [0, \infty)$ and Hermite $D = (-\infty, \infty)$. In particular, if we choose a set of Fourier functions like $\exp\left(i\frac{2\pi n}{L}x\right)$, where L is the fundamental period, it is possible to approximate any function $g(x) \in L^2(D)$ using a finite sum $\sum_n q_n \exp\left(i\frac{2\pi n}{L}x\right)$, and there is an analytical expression for $q_n = \frac{1}{L}\int_0^L g(x)\exp\left(-i\frac{2\pi n}{L}x\right)dx$ [23,24].

Finally, for a given set of well labeled values $\mathscr{D} = \{(x_m, y_m)\}_{m\in I_N}$ that come from a sample of a function $g(x)$, i.e. $y_m{=}g(x_m)$ and where the sampled values of x are equally distributed $x_{m+1} - x_m{=}\Delta$. Therefore with a feature map functions defined by $\phi_n(x_m) = \exp\left(i\frac{2\pi}{N}n{\cdot}m\right)$, where $(n, m) \in I_N^2$, thus the regression model can allow to obtain the discrete Fourier transform or `DFT` operator [26–28]. In this way the proposed ideas can be used to calculate the parameters of the regression model (19) as the discrete Fourier transform of the sampled vector $\boldsymbol{y}$. In the historical trajectory of scientific advancements, the significance of the discrete Fourier transform emerges prominently and in this work can be implemented in a classical or quantum way through a `QUBO-solver`.

5. Conclusions

In this work, we have addressed the training problem of certain `ML` model as a `QUBO` problem. The exploration of a binary-to-decimal encoding map published in [11], and our proposal inspired by one's complement, enhances data representation, significantly reducing the use of computational resources, so useful especially in the current context of quantum computing. The findings strongly indicate that incorporating quantum techniques in machine learning model training not only tackles the challenge of parameter explosion but also holds considerable promise for substantially accelerating the training process. By leveraging the distinctive properties of quantum computation, this approach emerges as a compelling avenue to overcome computational bottlenecks and usher in a new era of efficient machine learning model training.

Regarding the relationship between the decimal representation capability and the number of required boolean variables if $M_o/N_o \sim 1$ then $n_1/n_2 \sim 2$ and if $n_1/n_2 \sim 1$ then $N_o/M_o \sim 2^{L/2}$. The encoding `#2` has an advantage over encoding `#1` in that it requires fewer boolean variables, aligning with the number of logical qubits needed for resolution through `D-Wave`. Alternatively, for a regression problem with a dataset of size N, the encoding `#2` requires only half the number of logical qubits compared to encoding `#1`. On the other hand, encoding the sign of each decimal variable in a single bit assigns a large responsibility, because if any fluctuation of the qubo-solver were to change that bit the result would be catastrophic. This gives the encoding `#1` an advantage over the encoding `#2` with respect to robustness due to eventual fluctuations in the result of the optimization process.

This work has examined a regression model under a parametric and deterministic approach, while the conditions for a probabilistic approach to be accommodated with virtually the same mathematical resources have been discussed. In particular, $m > N$ will heavily overfiting, thus a form of regularization will be necessary [29]. Various regularization methods such as `Ridge-regression`, `LASSO` (least absolute shrinkage and selection operator), `Elastic-net`, `LARS` (least angle regression) and many others can be encompassed within the mathematical framework presented in this work [14]. The presented formalism could potentially be applied in other contexts within `ML`, such as the Bayesian paradigm. In such cases, concerning the optimization problem, information about the prior distribution of parameters $\boldsymbol{q}$ is typically incorporated into the distance function as an additive term, akin to the method of Lagrange multipliers [17].

The following three lines of work will be pursued as future work. Considering that the training procedure necessitates access to quantum hardware providers via the Internet, the inclusion of a security layer becomes imperative for users of such technologies. This ensures that users can be confident that their data, encoded in the $\boldsymbol{Q}_{\#}$ matrix sent over the network, remains uncompromised. In an upcoming article, we will address this topic.

It will also remain to study this alternative way of computing the Fourier transform and to establish, not only a comparison in terms of computational complexity with the flagship `FFT` [30] and `QFT` [31,32] algorithms, but also to assess the feasibility in terms of actual implementation of the `DFT` using `QUBO-solvers`.

Finally, it remains to study other `ML` models that can be included under the umbrella of this technique involving `QUBO-solvers`, such as classification, and even to address other algorithmic approaches [33–37].

Declarations

Not applicable

- Funding
- Ethics approval
- Consent to participate
- Consent for publication
- Code availability

Data availability

Data sharing not applicable to this article as no datasets were generated or analyzed during the current study. No data was used for the research described in the article.

Declaration of competing interest

The authors declare that there are no competing interests.

Acknowledgments

We thank UGR, FIDESOL, UNIR, and QCentroid for the support and recall also the anonymous readers for their constructive criticism of this work. This research has been partially supported by the project ECO-20241014: QUORUM funded by Ministerio de Ciencia, Innovación y Universidades, through CDTI and PID2022-138933OB-I00: ATQUE funded by MCIN/AEI/10.13039/501100011033/FEDER, EU.

References

[1] M. Merluzzi, P.D. Lorenzo, S. Barbarossa, Wireless edge machine learning: resource allocation and trade-Offs, IEEE Access 9 (2021) 45377–45398.
[2] J. Lee, Resource-efficient convolutional networks: a survey on model-, arithmetic-, and implementation-Level techniques, Assoc. Comput. Machin., 55 13s, 2023.
[3] S. Duda, P. Hofmann, N. Urbach, et al., The impact of resource allocation on the machine learning lifecycle, Bus. Inf. Syst. Eng. 66 (2024) 203–219.
[4] M.A. Kastner, Prospects for quantum dot implementation of adiabatic quantum computers for intractable problems, Proc. IEEE 93 10 (2005) 1765–1771.
[5] P. Date, R. Patton, C. Schuman, T. Potok, Efficiently embedding qubo problems on adiabatic quantum computers, Quantum Inf. Process 18 (2019) 117.
[6] M.S. Benkner, V. Golyanik, C. Theobalt, M. Moeller, Adiabatic quantum graph matching with permutation matrix constraints, in: 2020 International Conference on 3D Vision (3DV), Fukuoka, Japan, 2020, pp. 583–592.
[7] J.R. Jiang, C.W. Chu, Solving NP-hard problems with quantum annealing, in: 2022 IEEE 4th Eurasia Conference on IOT, Communication and Engineering (ECICE), Yunlin, Taiwan, 2022, pp. 406–411.
[8] J.D. Kelleher, M.B. Namee, A. D'Arcy, Fundamentals of Machine Learning for Predictive Data Analytics: Algorithms, Worked Examples, and Case Studies. The MIT Press, 2020.
[9] D. Poole, A. Mackworth, Artificial Intelligence: Foundations of Computational Agents, Cambridge University Press, 2010.
[10] A. Burkov, The hundred-page machine learning book (vol. 1), Andriy Burkov, Quebec City, QC, Canada, (2019).
[11] P. Date, D. Arthur, L. Pusey-Nazzaro, QUBO formulations for training machine learning models, Sci. Rep. 11 (2021) 10029.
[12] P. Date, T. Potok, Adiabatic quantum linear regression, Sci. Rep. 11 (2021) 21905.
[13] A. Kulkarni, D. Joshi, Quantum algorithm for linear regression for large feature-Set, in: 5th Joint International Conference on Data Science & Management of Data (9th ACM IKDD CODS and 27th COMAD) (CODS-COMAD 2022), New York, NY, USA, Association for Computing Machinery, 2022, pp. 316–317.
[14] F. Pedregosa, et al., Scikit-learn: machine learning in python, J. Mach. Lear. Res 12 (2011) 2825–2830. scikit-learn.org.
[15] D.-W. Systems Inc, Technical Description of the DWave Quantum Processing Unit, Technical Report, (D-Wave Systems Inc, Burnaby, BC, Canada) D-Wave User Manual 09-1109A-M, 2018.
[16] D.-W. Systems Inc, D-Wave Ocean SDK, github:dwave-ocean-sdk, 2018.
[17] T. Hastie, R. Tibshirani, J. Friedman, The Elements of Statistical Learning, New York Inc, Springer, 2001.
[18] M.P. Deisenroth, A.A. Faisal, C.S. Ong, Mathematics for Machine Learning, Cambridge University Press, 2020.
[19] R.A. Horn, C.R. Johnson, Matrix Analysis, Cambridge University Press, 2013. ISBN 978-0-521-54823-6.
[20] D.-W. Systems Inc, Solving Problems with Quantum Samplers, docs.dwavesys.com. https://docs.dwavequantum.com/docs/latest/c_gs_3.html
[21] M. Abramowitz, I.A. Stegun, Handbook of Mathematical Functions, Dover Publication Inc, 1974. ISBN 978-0-486-61272-0
[22] L.E. Franks, Signal Theory, Prentice-Hall, 1969. ISBN 978-0-138-100773.
[23] M. Reed, B. Simon, Methods of Modern Mathematical Physics, II of Fourier Analysis, Self-Adjointness, Academic Press, 1975. San Diego.
[24] R. Courant, D. Hilbert, Methods of Mathematical Physics, Vol. I, II, New York, John Wiley & Sons, 1953.
[25] C. Wang, H. Chen, E. Jonckheere, Quantum versus simulated annealing in wireless interference network optimization, Sci. Rep. 6 (2016) 25797.
[26] M.T. Heideman, D.H. Johnson, C.S. Burrus, Gauss and the history of the fast Fourier transform, Arch. Hist. Exact Sci. 34 (1985) 265–277. https://doi.org/10.1007/BF0034843
[27] A.V. Oppenheim, R.W. Schafer, J.R. Buck, Discrete-time Signal Processing, 2nd ed., Prentice Hal, 1999. ISBN 978-0-9745607-4-8.
[28] J.O. Smith, Mathematics of the Discrete Fourier Transform, W3K Publishing, 2007. ISBN 978-0-9745607-4-8
[29] P. Bühlmann, S.V.D. Geer, Statistics for High-Dimensional Data, Springer, 2011. https://doi.org/10.1007/978-3-642-20192-9
[30] J.W. Cooley, J.W. Tukey, An algorithm for the machine calculation of complex Fourier series, Math. Comput. 19 (90) (1965) 297–301. https://doi.org/10.2307/2003354
[31] D. Coppersmith, An approximate Fourier transform useful in quantum factoring, Technical Report RC19642, 1994.
[32] M. Nielsen, I. Chuang, Quantum Computation and Quantum Information, Cambridge University Press, 2000. ISBN 0-521-63503-9
[33] E. Farhi, J. Goldstone, S. Gutmann, A quantum approximate optimization algorithm, 2014 Preprint at, arXiv:1411.4028.
[34] M. Cerezo, A. Arrasmith, R. Babbush, et al., Variational quantum algorithms, Nat. Rev. Phys 3 (2021) 625–644. https://doi.org/10.1038/s42254-021-00348-9
[35] A.W. Harrow, A. Hassidim, S. Lloyd, Quantum algorithm for linear systems of equations, Phys. Rev. Lett 103 (2009) 150502.
[36] Y. Subasi, R.D. Somma, D. Orsucci, Quantum algorithms for systems of linear equations inspired by adiabatic quantum computing, Phys. Rev. Lett 122 (2019) 060504.
[37] J. Liu, M. Liu, J.P. Liu, et al., Towards provably efficient quantum algorithms for large-scale machine-learning models, Nat. Commun. 15 (2024) 434. https://doi.org/10.1038/s41467-023-43957-x